

EC2

Instance không SSH được – bạn debug theo thứ tự nào?

Một EC2 Linux đang chạy nhưng SSH timeout.

Bạn cần nói các bước thực tế như:

check Security Group port 22

check NACL

verify public IP / Elastic IP

check route table → Internet Gateway

dùng EC2 Instance Connect / Session Manager

check disk full → SSH fail

CPU EC2 luôn 90–100% → xử lý thế nào?

Không hỏi lý thuyết CloudWatch, mà hỏi:

bạn sẽ:

SSH vào → top / htop

check process

check memory swap

scale vertical hay horizontal?

attach Auto Scaling?

Ổ đĩa root full → production sắp down → làm gì ngay?

Expect:

extend EBS volume

resize filesystem (growpart + resize2fs / xfs_growfs)

clear log nếu emergency

Deploy app trên EC2 nhưng restart instance là mất data → vì sao?

Muốn bạn hiểu:

đang lưu trong instance storage

phải dùng EBS / EFS / S3

Bạn cần clone 10 server giống hệt server hiện tại → cách nhanh nhất?

Practical answer:

tạo AMI từ instance

launch từ AMI

Không phải manual setup.

Website trên EC2 bị chậm ngẫu nhiên mỗi ngày → kiểm tra gì?

Expect:

CPU credit (t2/t3 burst)

memory leak

disk IOPS bottleneck

network spike

Khi nào dùng Elastic IP? Và production cần lưu ý gì?

Practical point:

Elastic IP bị charge nếu không attach
nên gắn với NAT / bastion / fixed endpoint

Bạn cần login EC2 nhưng mất private key → xử lý thế nào?

Real-world skill:

detach root EBS
attach sang instance khác
sửa ~/.ssh/authorized_keys
hoặc dùng Session Manager nếu enabled.

EC2 boot fail sau khi update config → debug ra sao?

Expect:

dùng EC2 Serial Console
check system log
attach volume sang instance khác

Bạn cần zero-downtime deploy app trên EC2 → cách làm?

Possible answers:

load balancer + 2 instances
rolling replace
Auto Scaling group

Một EC2 private subnet cần internet outbound → làm sao?

Expect:

NAT Gateway / NAT instance
route table config

Làm sao giảm cost EC2 production ngay lập tức?

Practical:

resize instance type
dùng reserved / savings plan
stop unused instance
chuyển sang Graviton nếu phù hợp

RDS

RDS CPU 95% liên tục → bạn kiểm tra gì đầu tiên?

Expect workflow:

check slow queries

check active connections
check locks
check autovacuum running
check index missing
interviewer muốn thấy bạn vào:
`SELECT * FROM pg_stat_activity;`

RDS bị full storage → production sắp dừng → xử lý khẩn cấp?

Thực tế phải nói:
bật storage autoscaling nếu chưa
tăng allocated storage ngay
delete old logs / temp data
kiểm tra bloat tables
Không ai chờ migrate.

Query chạy rất nhanh trên local nhưng rất chậm trên RDS → vì sao?

Expect:
thiếu index production
data volume khác
parameter group khác
connection pooling
instance class yếu hơn

RDS connection full → app báo “too many connections” → làm gì?

Practical answer:
check `pg_stat_activity`
kill idle connections
bật connection pooler (PgBouncer)
tăng `max_connections` (cẩn thận memory)

Làm sao tìm query chậm nhất trong RDS PostgreSQL?

Expect:
bật extension `pg_stat_statements`
query:
`SELECT * FROM pg_stat_statements ORDER BY total_exec_time DESC;`

RDS suddenly restart → kiểm tra nguyên nhân ở đâu?

Expect:
RDS event log
CloudWatch metrics
maintenance window
storage full

memory exhausted

Bạn cần clone database production sang staging nhanh nhất?

Practical:

snapshot RDS

restore snapshot thành DB mới

Không dump/restore thủ công.

Deploy app mới → DB load tăng → scale RDS thế nào?

Expect:

Vertical scaling:

modify instance class

Read scaling:

tạo read replica

Transaction bị lock nhau → debug ra sao?

Expect SQL:

```
SELECT * FROM pg_locks;
```

hoặc:

```
SELECT pid, query FROM pg_stat_activity WHERE wait_event IS NOT NULL;
```

RDS query chậm dần theo thời gian → nguyên nhân phổ biến?

Practical causes:

table bloat

missing VACUUM

index fragmentation

growing dataset

Backup RDS nhưng muốn restore tới đúng 10:05 AM → làm sao?

Expect:

dùng Point-in-time recovery

Không phải snapshot thường.

App trong private subnet không connect được RDS → debug thế nào?

Expect checklist:

Security group inbound 5432

same VPC?

subnet route table

DNS endpoint đúng chưa

Lambda

Lambda bị timeout sau 30s → xử lý thế nào?

Expect workflow:

check CloudWatch log

tăng timeout nếu hợp lý

tối ưu query/API call

chuyển sang async processing nếu job dài

dùng SQS queue để tách workload

Lambda chạy chậm ở lần gọi đầu nhưng nhanh ở lần sau → vì sao?

interviewer muốn nghe:

cold start

package quá lớn

VPC attach gây delay

Practical fix:

giảm bundle size

provisioned concurrency

tránh VPC nếu không cần

Lambda connect RDS PostgreSQL nhưng thỉnh thoảng fail connection → vì sao?

Real-world issue:

Lambda scale → mở quá nhiều connections

RDS max_connections bị full

Practical solution:

dùng connection pooling

dùng RDS Proxy

Lambda xử lý file upload nhưng file 200MB → fail → làm sao?

Expect:

Lambda /tmp chỉ ~512MB–10GB

payload API Gateway limit

Correct design:

upload file → S3

trigger Lambda từ S3 event

Lambda chạy OK dev nhưng production bị out-of-memory → debug thế nào?

Expect:

check memory CloudWatch metric

tăng memory size

nhớ: memory ↑ → CPU cũng ↑

Bạn cần chạy job background 15 phút → Lambda phù hợp không?

Expect:

Lambda max 15 phút hard limit

nếu gần limit → nên dùng Step Functions / ECS / batch job
Interviewer muốn thấy bạn biết khi không nên dùng Lambda.

Lambda cần gọi API external nhưng đôi lúc API chậm → tránh retry storm thế nào?

Practical:

exponential backoff

timeout thấp

DLQ (dead-letter queue)

circuit breaker logic

Một Lambda bị trigger hàng nghìn lần trong vài giây → production DB bị chết → xử lý?

Expect:

throttle concurrency

reserved concurrency

buffer qua SQS

Deploy version mới Lambda nhưng muốn rollback nhanh → làm sao?

Correct:

dùng Lambda versions

dùng alias (prod → version)

Rollback chỉ đổi alias.

Lambda cần share code chung cho nhiều function → làm sao?

Expect:

Lambda Layer

Không copy-paste code.

Lambda log quá nhiều → CloudWatch cost tăng → xử lý?

Practical:

giảm log level

structured logging

set retention policy

API Gateway → Lambda → response đôi khi 502 → debug theo thứ tự nào?

Good answer order:

check Lambda log

check timeout mismatch

check response format JSON đúng chưa

check memory exhaustion

DynamoDB

Query DynamoDB rất chậm khi dữ liệu lớn → bạn kiểm tra gì?

Expect:

có đang dùng Scan không?

query đúng partition key chưa?

thiết kế key sai → hot partition?

interviewer muốn thấy bạn nghĩ tới data model trước, không phải tăng capacity.

Một partition DynamoDB bị throttling liên tục → vì sao?

Real-world cause:

traffic dồn vào 1 partition key

thiết kế key không phân tán

Practical fix:

thêm random suffix

redesign partition key

App cần query theo nhiều field khác nhau → DynamoDB xử lý sao?

Expect:

tạo GSI (Global Secondary Index)

không scan toàn table

DynamoDB suddenly cost tăng mạnh → debug theo hướng nào?

Practical checklist:

có Scan production không?

read/write capacity tăng?

item size lớn?

bật DynamoDB Streams mà không dùng?

Cần pagination API cho DynamoDB → làm thế nào?

Expect:

dùng LastEvaluatedKey

truyền ExclusiveStartKey cho query tiếp

Không dùng offset như SQL.

Insert batch 10k records → cách tối ưu?

Correct:

dùng BatchWriteItem

retry unprocessed items

Không loop insert từng record.

DynamoDB update bị conflict khi nhiều user update cùng lúc → xử lý?

Expect:

dùng conditional update
version attribute (optimistic locking)

Cần auto-expire session sau 24h → làm sao?

Expect:
TTL attribute DynamoDB
Không cần cron job delete.

Một API cần lấy user profile + order history → DynamoDB design ra sao?

Interviewer muốn thấy:
single-table design hoặc
partition key = user_id
sort key phân loại entity
Không join runtime.

DynamoDB table bị throttling nhưng CloudWatch thấy capacity còn → vì sao?

Real-world:
hot partition
burst limit hết
uneven traffic distribution

Cần backup DynamoDB trước khi deploy risky migration → cách nhanh nhất?

Expect:
on-demand backup
hoặc point-in-time recovery
Không export manual.

API đọc DynamoDB cực nhiều nhưng data ít thay đổi → giảm cost thế nào?

Practical:
bật DAX cache
hoặc cache Redis / memory layer

SQS

Message trong SQS không biến mất dù consumer đã xử lý → vì sao?

Expect:
quên delete message sau khi process
visibility timeout quá ngắn → message quay lại queue
đây là lỗi production rất phổ biến.

Consumer xử lý lâu → message bị chạy lại nhiều lần → fix thế nào?

Practical:
tăng visibility timeout

hoặc extend visibility timeout khi đang xử lý

Hệ thống suddenly xử lý trùng dữ liệu → SQS có thể là nguyên nhân không?

Expect:

Standard queue at-least-once delivery
message có thể duplicate

Fix:

idempotent processing
deduplication logic

Khi nào dùng FIFO queue thay vì Standard queue?

Real practical:

cần strict ordering

cần exactly-once processing (deduplication)

Nhưng:

throughput thấp hơn.

Message fail nhiều lần → bạn xử lý production thế nào?

Expect:

cấu hình DLQ (dead-letter queue)

set maxReceiveCount

Không retry vô hạn.

Queue backlog tăng rất nhanh → debug theo hướng nào?

Practical order:

check consumer còn chạy không

check processing time tăng không

check downstream DB/API slow

scale consumer

Làm sao scale consumer SQS tự động?

Expect:

Auto Scaling group theo queue depth

hoặc Lambda trigger SQS

Một message rất lớn (10MB) → gửi SQS fail → làm sao?

Expect:

SQS max 256KB

store payload trong S3

gửi reference qua SQS

Consumer crash giữa chừng → message có mất không?

Expect:
không mất
sau visibility timeout message quay lại queue
Interviewer muốn thấy bạn hiểu reliability model.

Queue có hàng trăm nghìn message cũ → xử lý nhanh production thế nào?

Practical:
scale consumer nhiều instance
tăng batch size receive
optimize processing logic

API gửi message SQS nhưng đôi khi user bấm 2 lần → tránh enqueue trùng?

Expect:
FIFO queue deduplication ID
hoặc app-level idempotency key

Làm sao biết message processing time trung bình đang tăng?

Expect:
CloudWatch metrics:
ApproximateAgeOfOldestMessage
queue depth
Không phải đo bằng log thủ công.

SNS

Publish SNS nhưng service downstream không nhận được message → debug theo thứ tự nào?

Expect flow:
check subscription confirmed chưa
check delivery status log
check permission IAM / resource policy
check endpoint sống không

SNS gửi HTTP webhook nhưng endpoint đôi khi down → xử lý production sao?

Practical:
SNS tự retry theo backoff
nên thêm DLQ cho subscription
endpoint phải idempotent

Khi nào dùng SNS + SQS chung?

Expect:
fan-out message
SNS publish → nhiều SQS consumer riêng

Đây là architecture rất phổ biến production.

Một subscriber xử lý chậm → có làm block subscriber khác không?

Correct:

không block (SNS push độc lập)

Interviewer muốn bạn hiểu SNS không phải queue.

SNS publish thành công nhưng Lambda không chạy → vì sao?

Expect:

Lambda permission chưa allow SNS invoke

subscription chưa confirm

region mismatch

filter policy chặn message

Một hệ thống cần gửi email + trigger Lambda + push queue cùng lúc → design sao?

Correct answer:

SNS topic

email subscription

Lambda subscription

SQS subscription

Message SNS quá lớn → publish fail → làm sao?

Expect:

SNS max ~256KB

lưu payload S3

publish link/reference

Subscriber chỉ muốn nhận message loại “order_created” → làm sao?

Expect:

dùng SNS filter policy theo message attributes

Không phải publish nhiều topic.

Làm sao tránh gửi email spam khi publish SNS nhiều lần?

Practical:

deduplicate ở app

hoặc throttle publish logic

SNS không tự chống duplicate

SNS publish nhưng cần đảm bảo message không mất → design thế nào?

Expect:

SNS → SQS subscription

queue giữ message bên
SNS đơn thuần không phải durable processing system.

Debug production thấy SNS retry webhook rất nhiều → nguyên nhân thường?

Real causes:

endpoint trả HTTP $\neq$ 2xx

timeout

TLS issue

DNS fail

Hệ thống event-driven cần broadcast event cho nhiều microservice → khi nào SNS tốt hơn chỉ dùng SQS?

Expect:

cần publish 1 lần → nhiều subscriber

dynamic subscribe/unsubscribe

SES

App gửi email qua SES nhưng user không nhận được → debug theo thứ tự nào?

Expect flow:

check SES sending statistics

check bounce / complaint

check spam folder

check domain verification

check DKIM/SPF config

SES báo gửi thành công nhưng email vào spam → production fix thế nào?

Practical:

verify domain (không chỉ verify email)

setup DKIM

setup SPF

tránh nội dung spam-like

warm-up sending volume

Muốn gửi 100k email marketing → tránh bị SES block sao?

Expect:

increase sending quota trước

gửi batch tăng dần (warm up)

dùng dedicated IP nếu volume lớn

xử lý bounce đúng cách

SES đang sandbox → production cần làm gì?

Expect:

request production access
verify domain
không chỉ verify email

Làm sao biết email nào bị bounce?

Correct:
dùng SNS notification cho SES events
subscribe Lambda / SQS / webhook
Không poll thủ công.

User unsubscribe nhưng hệ thống vẫn gửi mail → production fix sao?

Expect:
maintain suppression list nội bộ
check SES suppression list

Gửi email có attachment lớn → fail → vì sao?

Real-world:
SES max message size ~40MB (raw)
encode base64 làm size tăng
Fix:
upload file S3
gửi link download

Một email gửi rất chậm (vài phút mới tới inbox) → nguyên nhân thường?

Expect:
recipient mail server delay
SES reputation thấp
IP chưa warm-up
DNS config sai

Làm sao track user mở email hoặc click link?

Practical:
embed tracking pixel
dùng SES configuration set + event tracking
redirect link tracking

API gửi email thỉnh thoảng bị throttling → xử lý production sao?

Expect:
respect SES sending rate
queue email qua SQS
retry exponential backoff

Một bug khiến app gửi email duplicate → tránh sao?

Expect:

idempotency key

deduplicate ở DB

logging message ID

SES không tự chống duplicate.

Microservice cần gửi email async để không block request → design sao?

Good architecture:

API → push job vào SQS

worker/Lambda đọc queue → gửi SES

Không gửi email trực tiếp trong request.

AWS

Q1. What is AWS, and what are its Primary Services?

Xin chào và chào mừng bạn đến với buổi chia sẻ về những câu hỏi và câu trả lời phỏng vấn AWS hàng đầu do Intellipaat thực hiện. Hiện nay, 94% doanh nghiệp đang sử dụng một hình thức dịch vụ đám mây nào đó, trong khi 69% có kế hoạch mở rộng mức độ ứng dụng đám mây của mình, theo báo cáo của NASSCOM. Dựa trên doanh thu dịch vụ đám mây giai đoạn từ năm 2014 đến 2028, Ấn Độ được đánh giá có tiềm năng trở thành một trong những thị trường dịch vụ đám mây lớn nhất khu vực châu Á – Thái Bình Dương, bên cạnh Trung Quốc và Nhật Bản.

Mặc dù điện toán đám mây đã xuất hiện từ cuối những năm 1960, nhưng nó chỉ thực sự bắt đầu phát triển mạnh mẽ sau năm 2006 — cũng chính là thời điểm Amazon ra mắt AWS (Amazon Web Services). Đây không phải là sự trùng hợp ngẫu nhiên, bởi theo một báo cáo khác của NASSCOM, AWS hiện đang cung cấp các tùy chọn về năng lực tính toán và lưu trữ lớn nhất trên thị trường.

Chính vì lý do đó, số lượng việc làm dành cho kỹ sư AWS DevOps tại Ấn Độ được dự đoán sẽ vượt mốc 15.000 vào năm 2028. Tuy nhiên, đó mới chỉ là những dự báo cho tương lai. Còn ở hiện tại, bạn có thể tìm thấy hơn 9.000 cơ hội việc làm liên quan đến AWS trên LinkedIn, với mức lương trung bình khoảng 6 lakh rupee mỗi năm — một khởi đầu không hề tệ. Đáng chú ý là mức lương này hoàn toàn có thể tăng vọt lên hơn 30 lakh rupee mỗi năm sau 7 đến 9 năm kinh nghiệm.

Nếu tại thời điểm này AWS và ngành công nghiệp điện toán đám mây đã thu hút sự quan tâm của bạn, thì bạn đang ở đúng nơi. Như đã đề cập trước đó, trong nội dung này chúng tôi sẽ giới thiệu 30 câu hỏi phỏng vấn AWS phổ biến nhất, được tổng hợp cẩn thận dựa trên tần suất xuất hiện trong các buổi phỏng vấn thực tế. Các câu hỏi được chia thành ba cấp độ: dễ, trung bình và khó, mỗi cấp gồm 10 câu hỏi. Không cần nói thêm, hãy cùng bắt đầu.

Như đã đề cập, điện toán đám mây đã xuất hiện từ rất lâu, điều đó đồng nghĩa với việc trên thị trường có hàng nghìn lựa chọn khác nhau. Vì vậy, câu hỏi hiển nhiên đầu tiên là: AWS là gì và các dịch vụ chính của AWS là gì?

AWS (Amazon Web Services) là một nền tảng điện toán đám mây cho phép người dùng xây dựng, triển khai và quản lý hạ tầng trực tuyến mà không cần đến phần cứng tại chỗ (on-premise). Ngoài ra, AWS hoạt động theo mô hình trả tiền theo mức sử dụng (pay-as-you-go), cho phép người dùng chỉ trả chi phí cho những tài nguyên mà họ thực sự sử dụng. Một số dịch vụ nổi bật của AWS bao gồm EC2 và Lambda trong mảng tính toán, S3 và EBS trong lưu trữ, Amazon RDS và DynamoDB cho cơ sở dữ liệu, cùng nhiều dịch vụ khác như Route 53, IAM, SageMaker, v.v.

Tuy nhiên, cần lưu ý rằng trong buổi phỏng vấn, bạn chỉ nên nhắc đến những dịch vụ mà bạn thực sự quen thuộc. Việc đề cập đến các dịch vụ chưa nắm rõ có thể khiến bạn rơi vào tình huống bất lợi, bởi có đến 80% khả năng câu hỏi tiếp theo của nhà tuyển dụng sẽ xoay quanh chính những dịch vụ mà bạn vừa nhắc tới — và đó cũng là lý do dẫn chúng ta đến câu hỏi tiếp theo.

Q2. Explain The Key Components of Amazon S3

Amazon S3 là một dịch vụ lưu trữ có khả năng mở rộng do AWS cung cấp. Dữ liệu được lưu trữ bên trong các bucket, và mỗi bucket đều có một tên duy nhất trên phạm vi toàn cầu, không chỉ riêng trong tài khoản Amazon của bạn.

Tiếp theo là dữ liệu được lưu trong các bucket, được gọi là object. Mỗi object bao gồm dữ liệu thực tế và metadata (siêu dữ liệu) mô tả thông tin về dữ liệu đó. Ngoài ra, mỗi object còn có một key, đóng vai trò như định danh duy nhất cho object trong bucket và hoạt động giống như một đường dẫn trỏ đến object.

Bên cạnh đó, bucket S3 còn hỗ trợ versioning để khôi phục các phiên bản object khi cần, cũng như mã hóa dữ liệu ở cả phía máy chủ (server-side) và phía khách (client-side) nhằm đảm bảo an toàn cho dữ liệu. Tiếp theo, chúng ta sẽ chuyển sang câu hỏi kế tiếp.

Q3. What are the different types of cloud computing models?

Có ba mô hình điện toán đám mây chính: IaaS (Infrastructure as a Service – Hạ tầng dưới dạng dịch vụ), PaaS (Platform as a Service – Nền tảng dưới dạng dịch vụ) và SaaS (Software as a Service – Phần mềm dưới dạng dịch vụ).

Infrastructure as a Service (IaaS) giống như việc thuê một mảnh đất trống. Bạn phải tự xây dựng ngôi nhà của mình, bao gồm làm móng, xây tường, lắp đặt hệ thống ống nước, điện và mọi thứ khác. Nói cách khác, nhà cung cấp chỉ cung cấp hạ tầng cơ bản, còn bạn

phải xây dựng mọi thứ từ đầu. Ví dụ cho mô hình này là Amazon EC2 hay Google Compute Engine.

Platform as a Service (PaaS) giống như việc thuê một ngôi nhà đã được xây sẵn. Bạn chỉ cần tập trung vào việc thiết kế bên trong như sơn tường, bố trí nội thất mà không cần lo lắng về hạ tầng phía dưới như điện hay nước. Bạn chỉ quan tâm đến cách sử dụng và phát triển ứng dụng trên nền tảng đó. Ví dụ tiêu biểu là AWS Elastic Beanstalk hay Heroku.

Cuối cùng, Software as a Service (SaaS) giống như việc chuyển vào ở trong một ngôi nhà đã được trang bị đầy đủ nội thất. Mọi thứ như đồ đạc, thiết bị nhà bếp, tiện ích và thậm chí cả trang trí đều đã sẵn sàng. Bạn chỉ cần sử dụng ngay khi bước vào. Ví dụ cho mô hình này là Microsoft Office 365 hay Zoom.

EC2 là một trong những chủ đề phổ biến nhất khi nói về AWS, và khi bạn đã nhắc đến nó hai lần, điều đó tự nhiên sẽ khiến nhà tuyển dụng tò mò về mức độ hiểu biết của bạn về EC2. Điều này dẫn chúng ta đến câu hỏi tiếp theo.

Q4. What do you know about Amazon EC2?

Amazon EC2, ở mức cơ bản nhất, giống như việc thuê một chiếc máy tính ảo trên nền tảng đám mây. Thay vì phải thiết lập và duy trì một máy tính vật lý, bạn có thể thuê một máy chủ ảo từ Amazon bất cứ khi nào cần. Bạn có thể lựa chọn cấu hình, kích thước và sức mạnh của máy theo nhu cầu, sử dụng trong khoảng thời gian mong muốn và chỉ phải trả tiền cho đúng thời gian bạn sử dụng. Khi không còn cần nữa, bạn có thể tắt hoặc xóa máy đó. Sau phần giải thích đơn giản này, nhà phỏng vấn thường sẽ muốn kiểm tra xem bạn đã hiểu sâu đến mức nào về các khái niệm này hoặc liệu bạn đã từng áp dụng kiến thức lý thuyết vào thực tế hay chưa, và vì vậy họ sẽ dẫn đến câu hỏi tiếp theo.

Q5. What is the difference between stopping and terminating an EC2 instance?

Khi bạn dừng (stop) một EC2 instance, máy chủ sẽ được tắt nhưng không bị xóa vĩnh viễn. Điều này có nghĩa là bạn chỉ đang tạm thời dừng hệ thống và có thể khởi động lại nó bất cứ lúc nào sau đó, tuy nhiên địa chỉ IP công khai có thể sẽ thay đổi. Ngược lại, khi bạn chấm dứt (terminate) một EC2 instance, instance đó sẽ bị xóa hoàn toàn. Tài nguyên tính toán và lưu trữ cũng được giải phóng, đồng nghĩa với việc toàn bộ dữ liệu lưu trên instance sẽ bị mất vĩnh viễn. Nói một cách đơn giản, stop giống như tạm dừng, còn terminate tương đương với xóa. Sau câu hỏi này, người phỏng vấn thường sẽ hỏi thêm về khái niệm regions và availability zones trong EC2, những thông tin mà bạn cũng có thể thấy ngay khi tạo một EC2 instance trong tài khoản AWS của mình, và từ đó dẫn sang câu hỏi tiếp theo.

Q6. Explain the concept of availability zones and regions in AWS

Trong AWS, Region có thể được hiểu là một khu vực địa lý tách biệt, nơi AWS triển khai hạ tầng của mình; bạn có thể hình dung nó giống như một thành phố lớn trong một quốc gia. Ví dụ, nếu bạn muốn ứng dụng hoặc website của mình đặt gần người dùng tại Ấn Độ, bạn có thể chọn một region nằm trong quốc gia này, chẳng hạn như Asia Pacific (Mumbai). Trong khi đó, Availability Zone (AZ) là các vị trí độc lập nằm bên trong một region; bạn có thể xem các availability zone giống như những khu dân cư trong cùng một thành phố. Nếu xảy ra sự cố ở một khu vực, các khu vực khác vẫn có thể tiếp tục vận hành máy chủ bình thường. Hiện nay, AWS có tổng cộng 34 region địa lý trên toàn thế giới và 108 availability zone. Một câu hỏi thường gặp tiếp theo là AWS quản lý số lượng máy chủ khổng lồ này như thế nào, và câu trả lời sẽ nằm ở phần tiếp theo.

Q7. Define "Elastic Load Balancer" in AWS

Elastic Load Balancer trong AWS là một dịch vụ có nhiệm vụ phân phối lưu lượng truy cập của ứng dụng đến nhiều mục tiêu (targets) khác nhau, giúp hệ thống luôn sẵn sàng hoạt động và đạt tính sẵn sàng cao (high availability). ELB đảm bảo rằng nếu một máy chủ gặp sự cố, lưu lượng truy cập sẽ tự động được chuyển hướng sang các máy chủ khác, từ đó tăng khả năng chịu lỗi (fault tolerance). Ngoài ra, ELB còn thường xuyên kiểm tra tình trạng hoạt động (health check) của các máy chủ, bảo đảm rằng chỉ những máy chủ đang hoạt động tốt mới nhận được lưu lượng truy cập. Đó là những kiến thức cơ bản cần biết về Elastic Load Balancer, và tiếp theo chúng ta sẽ chuyển sang câu hỏi kế tiếp.

Q8. What are the differences between horizontal scaling and vertical scaling?

Cả mở rộng theo chiều ngang (horizontal scaling) và mở rộng theo chiều dọc (vertical scaling) đều là những chiến lược được sử dụng để quản lý và cải thiện hiệu năng cũng như khả năng đáp ứng của hệ thống khi khối lượng công việc tăng lên. Mở rộng theo chiều ngang là việc thêm hoặc bớt các tài nguyên, chẳng hạn như các instance EC2, để phân phối tải công việc một cách đồng đều. Trong khi đó, mở rộng theo chiều dọc là việc tăng hoặc giảm năng lực của từng tài nguyên riêng lẻ nhằm nâng cao sức mạnh của phần cứng hoặc instance hiện có bằng cách nâng cấp cấu hình. Sau đây, chúng ta sẽ chuyển sang câu hỏi áp chót.

Q9. What is AWS IAM, and why is it important?

AWS IAM (Identity and Access Management) là một dịch vụ của AWS cho phép kiểm soát an toàn việc truy cập vào các tài nguyên AWS. Nói một cách đơn giản, IAM xác định “ai” (người dùng, nhóm hoặc vai trò) và “cái gì” mà họ được phép truy cập, chẳng hạn như các dịch vụ và tài nguyên AWS cụ thể. Một trong những lý do khiến IAM trở nên phổ biến là khả năng cung cấp quyền truy cập chi tiết (granular), ví dụ cho phép một instance EC2 truy cập vào một bucket S3 nhưng không được truy cập vào RDS. Ngoài ra, IAM còn có khả năng cấp quyền truy cập tạm thời vào các dịch vụ AWS cho người dùng, ứng dụng hoặc thậm chí các tài nguyên AWS. Cuối cùng, chúng ta đã đến câu hỏi cuối cùng của mô-đun này.

Q10. What are Security Groups and how do they differ from Network ACLs?

Để hiểu Network ACLs, hãy tưởng tượng bạn đang tổ chức một buổi tiệc ở văn phòng tại cấp độ tầng. Ban quản lý tòa nhà chỉ cho phép nhân viên trong công ty bạn vào tòa nhà, nghĩa là bất kỳ ai không thuộc công ty sẽ bị chặn ngay từ cổng chính. Đối với Security Groups, căn phòng nơi bạn tổ chức buổi tiệc được bảo vệ bởi một nhân viên an ninh, và chỉ những phòng ban cụ thể như IT hoặc Marketing mới được phép vào phòng tiệc, ngay cả khi họ đang ở cùng tầng. Cơ chế bảo mật hai lớp này đảm bảo rằng dù ai đó có lọt được vào tầng hay subnet thì họ vẫn cần có quyền để vào phòng tiệc, hay nói cách khác là vào instance, từ đó tạo ra sự bảo vệ nhiều tầng.

Xét về sự khác biệt, Security Groups hoạt động ở cấp độ “phòng”, đảm bảo bảo vệ từng instance riêng lẻ, tương tự như phòng tiệc. Network ACLs thì áp dụng cho toàn bộ subnet, kiểm soát tất cả các “phòng” trên cùng một tầng bằng cách gác cổng ngay từ lối vào chính. Một khác biệt quan trọng nữa là Security Groups có trạng thái (stateful), còn Network ACLs thì không có trạng thái (stateless). Điều này có nghĩa là Security Groups ghi nhớ những kết nối mà chúng đã cho phép, trong khi Network ACLs không nhớ ai đã được phép đi qua; mỗi lần có lưu lượng đi qua cổng, các quy tắc đều được kiểm tra lại từ đầu, kể cả khi lưu lượng đó đang rời khỏi hệ thống. Với nội dung này, chúng ta đã kết thúc mô-đun dễ và sẽ chuyển sang mô-đun tiếp theo – mô-đun trung bình – bắt đầu với câu hỏi thứ 11.

Q11. How does Amazon Route 53 ensure high availability and low latency?

Amazon Route 53 là một dịch vụ DNS (Domain Name System – hệ thống phân giải tên miền). Mục đích của DNS là chuyển đổi các tên miền mà con người có thể đọc được thành địa chỉ IP mà máy tính có thể hiểu. Nếu nghe có vẻ trừu tượng, hãy xem một ví dụ: giả sử một người muốn truy cập trang web amazon.com. Tên miền amazon.com có nhiều địa chỉ IP tương ứng, đại diện cho nhiều máy chủ khác nhau. Vậy làm sao người dùng biết nên kết nối tới máy chủ nào? Lúc này, Amazon Route 53 sẽ phát huy vai trò của mình bằng cách chuyển đổi tên miền dễ đọc như “amazon.com” thành một địa chỉ IP cụ thể, từ đó giúp xác định máy chủ cần kết nối. Đây là một trong những dịch vụ cốt lõi của Amazon Route 53: một hệ thống DNS do AWS cung cấp, được thiết kế để ánh xạ tên miền sang địa chỉ IP.

Ngoài chức năng DNS cơ bản, Route 53 còn hỗ trợ nhiều tính năng khác để đảm bảo tính sẵn sàng cao và độ trễ thấp. Trong thực tế, một website thường có nhiều máy chủ và nhiều địa chỉ IP, tương tự như mô hình CDN (Content Delivery Network). Tất cả các địa chỉ IP này đều trở về cùng một website. Route 53 có khả năng định tuyến người dùng đến máy chủ gần nhất và có mức tải thấp hơn. Ví dụ, nếu một hệ thống có nhiều máy chủ với số lượng người dùng khác nhau, Route 53 có thể tự động chọn máy chủ đang có ít người truy cập nhất, giúp việc kết nối trở nên nhanh và mượt hơn. Bên cạnh đó, Amazon Route 53 còn sử dụng cơ chế định tuyến dựa trên độ trễ (latency-based

routing), đưa yêu cầu của người dùng tới điểm cuối có độ trễ mạng thấp nhất so với vị trí của họ, tức là máy chủ gần nhất và ít bị quá tải. Nhờ vậy, dữ liệu được truy xuất nhanh hơn và người dùng có trải nghiệm liền mạch hơn. Sau khi hiểu được cơ chế này, chúng ta sẽ tiếp tục với câu hỏi tiếp theo.

Q12. Describe the key features of AWS Lambda

AWS Lambda là một trong những khái niệm rất thú vị của AWS. Để dễ hiểu, hãy hình dung AWS Lambda giống như việc bạn thuê một freelancer làm việc cho công ty mình. Freelancer này chỉ làm việc khi có yêu cầu từ khách hàng. Ví dụ, khi một người dùng đăng ký nhận bản tin hoặc đăng ký trên website của bạn, lúc đó bạn mới gửi cho họ một email xác nhận hoặc email cảm ơn vì đã đăng ký. Hành động đăng ký của người dùng chính là “trigger” (trình kích hoạt). Dựa trên trigger này, AWS Lambda – giống như freelancer – sẽ tự động thực hiện nhiệm vụ, chẳng hạn như gửi email cho khách hàng.

Về bản chất, bạn chỉ cần thiết lập Lambda và xác định các trigger phát sinh từ hành động của người dùng, sau đó Lambda sẽ tự động phản hồi. Một điểm thú vị nữa là Lambda không “ngồi chờ” sự kiện xảy ra. Nó chỉ chạy khi có trigger, vì vậy bạn chỉ phải trả tiền cho thời gian thực thi, tài nguyên tính toán và lưu trữ mạng mà Lambda thực sự sử dụng tại thời điểm đó. Nói theo thuật ngữ kỹ thuật, AWS Lambda là một dịch vụ tính toán không máy chủ (serverless) cho phép bạn chạy code mà không cần cấp phát hay quản lý máy chủ. AWS sẽ tự động quản lý hạ tầng phía sau, và Lambda chỉ kích hoạt để xử lý khi có sự kiện hoặc trigger xảy ra.

Q13. What is AWS CloudFormation, and what are its benefits?

AWS CloudFormation, hiểu một cách đơn giản nhất, giống như một bản mẫu (template). Vậy tại sao chúng ta cần một bản mẫu? Hãy tưởng tượng có một đội xây dựng phải xây một ngôi nhà. Họ xây ngẫu hứng và ngôi nhà đó lại rất đẹp. Mọi người xung quanh đều muốn có một ngôi nhà y hệt như vậy. Tuy nhiên, vì đội xây dựng không có bản vẽ hay kế hoạch nào, nên khi họ xây những ngôi nhà tiếp theo, kết quả sẽ không thể giống hệt ngôi nhà đầu tiên. Ngược lại, nếu ngay từ đầu họ có một bản thiết kế (blueprint), thì họ có thể dùng chính bản thiết kế đó để xây các ngôi nhà khác, và tất cả đều sẽ giống nhau. Đây chính là lý do chúng ta cần một template, và đó cũng là vai trò của AWS CloudFormation.

Trong bối cảnh AWS, hãy tưởng tượng bạn có một hệ thống gồm Lambda, RDS, S3 bucket và các IAM user. Mỗi thành phần đều có những cấu hình riêng: Lambda có cấu hình trigger và cách phản hồi, S3 bucket có các thiết lập cụ thể, RDS cần chọn loại cơ sở dữ liệu và thông số, còn IAM user thì cần các policy phù hợp. Việc nhớ chính xác toàn bộ những cấu hình này là rất khó. AWS CloudFormation giải quyết vấn đề đó bằng cách cho phép bạn tạo ra một template chứa toàn bộ hạ tầng và cấu hình. Template này có thể được tái sử dụng nhiều lần để tạo ra nhiều bản sao giống hệt nhau.

Về mặt kỹ thuật, AWS CloudFormation là một dịch vụ cho phép bạn định nghĩa và quản lý hạ tầng AWS dưới dạng mã (Infrastructure as Code). Bạn tạo các template bằng định dạng JSON hoặc YAML để mô tả các tài nguyên AWS và cấu hình cần thiết cho ứng dụng. Có nhiều cách để tạo template CloudFormation: bạn có thể viết trực tiếp bằng code, hoặc sử dụng công cụ thiết kế trực quan do AWS cung cấp để thiết kế hạ tầng và dịch vụ, sau đó xuất ra template.

Q14. Explain the difference between Amazon SQS and Amazon SNS

Amazon SQS có thể được hình dung giống như khi bạn nhắn tin với một người bạn rất thân trên WhatsApp. Bạn gửi rất nhiều tin nhắn, và tất cả các tin nhắn này được lưu trong một hàng đợi (queue) theo nguyên tắc FIFO – First In, First Out, tức là tin nhắn nào đến trước sẽ được xử lý trước. Amazon SQS chính là dịch vụ đảm bảo các thông điệp được lưu trữ an toàn trong hàng đợi và được chuyển tới người nhận một cách đáng tin cậy, theo cơ chế xử lý bất đồng bộ. Mỗi thông điệp từ người gửi sẽ được đưa vào queue, sau đó lần lượt chuyển tới người nhận theo đúng thứ tự.

Ngược lại, Amazon SNS hoạt động giống như một kênh phát sóng. Nếu SQS là mô hình một-một, thì SNS là một-nhiều. Hãy tưởng tượng một chương trình radio: bất kỳ ai đã đăng ký và đang nghe kênh đó đều có thể nhận được thông điệp được phát. Amazon SNS (Simple Notification Service) là một dịch vụ nhắn tin theo mô hình publish-subscribe, cho phép ứng dụng hoặc các microservice giao tiếp bằng cách phát thông điệp tới nhiều subscriber cùng lúc.

Về sự khác biệt chính, Amazon SQS là dịch vụ dựa trên hàng đợi, tập trung vào việc đảm bảo thông điệp được lưu trữ và chuyển giao an toàn, với thời gian lưu trữ thông điệp lên đến 14 ngày. Trong khi đó, Amazon SNS không chú trọng vào việc lưu trữ lâu dài mà tập trung vào việc phát thông điệp đến tất cả những ai đã đăng ký kênh. Chính vì vậy, trong thực tế SQS thường được kết hợp với SNS để đảm bảo rằng khi một thông điệp được broadcast, mỗi subscriber đều nhận được thông điệp đó một cách đáng tin cậy.

Q15. What is Amazon RDS, and when should it be used?

Amazon RDS (Relational Database Service) là một dịch vụ cơ sở dữ liệu quan hệ được AWS quản lý hoàn toàn, giúp đơn giản hóa việc thiết lập, vận hành và mở rộng cơ sở dữ liệu trên nền tảng đám mây. RDS hỗ trợ nhiều hệ quản trị cơ sở dữ liệu phổ biến như MySQL, PostgreSQL, MariaDB, Oracle và SQL Server. Có thể bạn sẽ thắc mắc vì sao cần dùng cơ sở dữ liệu trên đám mây trong khi vẫn có thể cài MySQL và sử dụng trên máy tính của mình. Lý do nằm ở khối lượng công việc và mức độ phức tạp khi quản lý cơ sở dữ liệu cho các website quy mô lớn như trang thương mại điện tử hay hệ thống có lượng truy cập cao.

Khi tự quản lý cơ sở dữ liệu, bạn phải tự tay cập nhật và vá lỗi mỗi khi có bản cập nhật mới, tự tạo bản sao lưu và bản dự phòng để bảo vệ dữ liệu người dùng, tự mở rộng hệ

thông khi lưu lượng truy cập tăng đột biến (thậm chí có thể xảy ra vào nửa đêm), liên tục giám sát để phát hiện các hoạt động bất thường, phòng chống tấn công như SQL injection, và đảm bảo tính sẵn sàng cao bằng cách tự cấu hình lại hoặc chuyển hướng lưu lượng khi một máy chủ gặp sự cố. Với website nhỏ, những việc này có thể còn kiểm soát được, nhưng với hệ thống lớn thì chúng nhanh chóng trở nên phức tạp, căng thẳng và tốn nhiều nguồn lực.

Đó chính là lúc Amazon RDS phát huy giá trị. RDS tự động đảm nhiệm các công việc như vá lỗi, cập nhật, sao lưu, mở rộng, giám sát và đảm bảo tính sẵn sàng cao, giúp bạn tập trung vào phát triển ứng dụng thay vì lo lắng về vận hành cơ sở dữ liệu.

Q16. What are the core components of the AWS Well-Architected Framework?

AWS Well-Architected Framework là một tập hợp các nguyên tắc, hướng dẫn và thực tiễn tốt nhất, được thiết kế để giúp kiến trúc sư và lập trình viên xây dựng các hệ thống an toàn, hiệu năng cao, linh hoạt, bền bỉ và hiệu quả về chi phí trên nền tảng AWS. Nói một cách đơn giản, framework này hướng dẫn bạn cách thiết kế và vận hành hạ tầng cũng như các dịch vụ trên AWS sao cho tận dụng tối đa tài nguyên, tối ưu hiệu suất và mang lại trải nghiệm liền mạch cho cả người dùng lẫn đội ngũ vận hành.

Khi thiết kế hệ thống trên AWS, bạn cần cân nhắc nhiều yếu tố như mô hình chi phí, loại và số lượng máy chủ phù hợp với lưu lượng truy cập, cũng như cách tối ưu từng dịch vụ để đạt hiệu quả cao nhất với chi phí thấp nhất. AWS Well-Architected Framework xoay quanh 5 trụ cột chính:

Operational Excellence (Vận hành xuất sắc): tập trung vào giám sát, tự động hóa và cải tiến liên tục. AWS là nền tảng đám mây tự động hóa cao, giúp bạn giảm thiểu các thao tác thủ công. Các dịch vụ giám sát (như GuardDuty) có thể theo dõi hệ thống, phát hiện hành vi bất thường, kiểm soát ngưỡng tài nguyên như CPU, RAM và gửi cảnh báo khi có sự cố.

Security (Bảo mật): bảo vệ dữ liệu, hệ thống và tài sản bằng cách áp dụng các thực tiễn bảo mật tốt nhất. AWS IAM cho phép bạn kiểm soát truy cập chi tiết đến từng tài nguyên, quy định ai được truy cập, được làm gì và ở mức độ nào, từ đó đảm bảo an toàn cho hệ thống.

Reliability (Độ tin cậy): đảm bảo hệ thống có khả năng phục hồi nhanh chóng khi xảy ra sự cố. Ví dụ, sử dụng Amazon RDS giúp tự động chuyển đổi sang cơ sở dữ liệu dự phòng hoặc tạo phiên bản mới khi một máy chủ gặp lỗi, giảm thiểu hoặc tránh hoàn toàn thời gian gián đoạn.

Performance Efficiency (Hiệu quả hiệu năng): sử dụng tài nguyên đám mây một cách hiệu quả để đáp ứng nhu cầu hệ thống. Khi cấu hình các dịch vụ như EC2, bạn cần lựa chọn cấu hình phù hợp với yêu cầu thực tế, đồng thời tận dụng các dịch vụ hỗ trợ mở rộng và quản lý như Elastic Beanstalk.

Cost Optimization (Tối ưu chi phí): kiểm soát và giảm thiểu chi phí bằng cách chỉ sử dụng tài nguyên khi cần thiết. Ví dụ, AWS Lambda chỉ chạy khi có sự kiện kích hoạt, giúp tránh lãng phí tài nguyên tính toán và mạng, từ đó tiết kiệm đáng kể ngân sách.

Nhờ tuân theo các trụ cột này, AWS Well-Architected Framework giúp bạn xây dựng hệ thống vừa ổn định, an toàn, vừa hiệu quả và tiết kiệm chi phí.

Q17. Explain the concept of Elastic IP in AWS

Để hiểu Elastic IP, hãy tưởng tượng bạn có một máy chủ chính và có nhiều máy tính, mỗi máy sử dụng một địa chỉ IP để kết nối đến máy chủ đó. Nếu có sự cố xảy ra và bạn phải khởi động lại toàn bộ máy chủ, địa chỉ IP ban đầu có thể bị thay đổi. Khi đó, tất cả các máy tính đang kết nối sẽ phải tìm và kết nối lại với địa chỉ IP mới, điều này có thể gây ra thời gian gián đoạn lớn cho website hoặc ứng dụng của bạn.

Để tránh tình trạng downtime này, AWS cung cấp Elastic IP. Elastic IP là một địa chỉ IPv4 công khai tĩnh do AWS cấp, có thể gán cho các tài nguyên AWS như một phiên bản Amazon EC2. Ngay cả khi máy chủ gặp sự cố hoặc bị khởi động lại, địa chỉ IP này vẫn không thay đổi và tiếp tục gán với tài nguyên đó. Nói cách khác, Elastic IP đảm bảo việc giao tiếp ổn định và nhất quán trên Internet, giúp hệ thống của bạn luôn sẵn sàng hoạt động mà không bị gián đoạn do thay đổi địa chỉ IP.

Q18. What is AWS Elastic Beanstalk, and how does it simplify application deployment?

AWS Elastic Beanstalk là một dịch vụ **Platform as a Service (PaaS)** giúp đơn giản hóa quá trình triển khai, quản lý và mở rộng các ứng dụng web cũng như dịch vụ. Để dễ hình dung, hãy tưởng tượng có một đầu bếp rất giỏi nấu ăn – trong thực tế, đó giống như một lập trình viên giỏi viết code. Người đầu bếp này muốn mở nhà hàng mơ ước của mình, nhưng việc xây dựng nhà hàng (cơ sở hạ tầng) không phải là chuyên môn của anh ta. Thay vì tự làm tất cả, anh ta thuê một đội thợ xây dựng để lo phần thi công, còn bản thân chỉ tập trung vào nấu ăn và phục vụ khách hàng.

Elastic Beanstalk hoạt động theo cách tương tự. Bạn chỉ cần tập trung vào việc viết và duy trì mã nguồn ứng dụng, ví dụ như ứng dụng Node.js, rồi tải code đó lên Elastic Beanstalk. Dịch vụ này sẽ tự động lo toàn bộ phần phía sau như triển khai ứng dụng, cấu hình máy chủ, quản lý cơ sở hạ tầng, cấp địa chỉ IP, tích hợp với Route 53, cũng như tự động mở rộng hoặc thu hẹp tài nguyên (server, database) dựa trên lưu lượng truy cập.

Elastic Beanstalk đảm bảo ứng dụng của bạn luôn sẵn sàng hoạt động và có khả năng xử lý tốt lượng truy cập thay đổi, trong khi bạn chỉ cần đảm bảo code chạy ổn định và không có lỗi.

Q19. What are the main features of Amazon DynamoDB?

Amazon DynamoDB là một dịch vụ cơ sở dữ liệu khác do Amazon cung cấp. Trước đó chúng ta đã nói về Amazon RDS – một cơ sở dữ liệu quan hệ (SQL). Trong khi đó, DynamoDB giống như MongoDB, tức là một cơ sở dữ liệu NoSQL. Tuy là NoSQL, DynamoDB vẫn chia sẻ nhiều đặc điểm quan trọng với RDS như khả năng mở rộng, vá lỗi, cập nhật và quản lý tự động. Ở góc độ kỹ thuật, Amazon DynamoDB là một dịch vụ cơ sở dữ liệu NoSQL được quản lý hoàn toàn, được thiết kế để cung cấp hiệu năng nhanh và ổn định cùng với khả năng mở rộng liền mạch.

Những tính năng nổi bật của DynamoDB bao gồm: hệ thống fully managed và serverless, độ trễ thấp với hiệu năng cao, khả năng mở rộng tự động, tính sẵn sàng toàn cầu và thiết kế schema linh hoạt. Việc fully managed và serverless có nghĩa là AWS tự động xử lý toàn bộ phần cấp phát phần cứng, vá lỗi phần mềm, thiết lập, sao lưu và mở rộng – tương tự như RDS. DynamoDB cung cấp độ trễ ở mức mili-giây đơn lẻ, cho tốc độ truy xuất dữ liệu rất nhanh. Dịch vụ này có khả năng tự động mở rộng lên hoặc thu hẹp xuống để xử lý khối lượng dữ liệu và số lượng request lớn, đồng thời hỗ trợ Global Tables để sao chép dữ liệu đa vùng (multi-region). Là một cơ sở dữ liệu NoSQL, DynamoDB có schema linh hoạt, không quá chặt chẽ như RDS. Vì vậy, nếu bạn cần lưu trữ và truy cập nhanh khối lượng dữ liệu lớn, DynamoDB là lựa chọn phù hợp; còn nếu bạn cần dữ liệu được tổ chức chặt chẽ với khóa chính, khóa ngoại và tuân theo các thuộc tính ACID, thì RDS sẽ là lựa chọn thích hợp hơn.

Q20. How does Amazon VPC enhance security in AWS environments?

Amazon VPC (Virtual Private Cloud) giúp tăng cường bảo mật trong môi trường AWS bằng cách cho phép bạn tạo ra một không gian mạng riêng, tách biệt và an toàn bên trong hạ tầng đám mây của AWS. Trong VPC có hai thành phần quan trọng là public subnet và private subnet. Để dễ hiểu, hãy tưởng tượng bạn có một máy chủ chạy website chính, trong đó có phần frontend và backend. Frontend là phần giao diện hiển thị cho người dùng bên ngoài – bất kỳ máy tính hay địa chỉ IP nào truy cập vào website đều có thể nhìn thấy, nên nó được đặt trong public subnet. Ngược lại, backend thường là cơ sở dữ liệu và các dịch vụ xử lý dữ liệu người dùng, đây là thông tin nhạy cảm và không thể công khai, vì vậy nó được đặt trong private subnet và không cho truy cập trực tiếp từ bên ngoài.

Về mặt kỹ thuật, Amazon VPC cho phép bạn định nghĩa toàn bộ môi trường mạng của mình như dải địa chỉ IP, subnet, bảng định tuyến và các chính sách bảo mật, từ đó cô lập tài nguyên và kiểm soát chặt chẽ việc truy cập. Ví dụ, khi bạn triển khai ứng dụng bằng

AWS Elastic Beanstalk, AWS sẽ cung cấp một VPC mặc định để đảm bảo rằng các máy chủ và cơ sở dữ liệu chạy ở backend được bảo vệ an toàn, trong khi frontend vẫn có thể phục vụ người dùng bình thường. Nhờ việc tách biệt rõ ràng giữa public subnet và private subnet, Amazon VPC giúp giảm thiểu rủi ro bảo mật và đảm bảo dữ liệu quan trọng luôn được bảo vệ.

Q21. What is the AWS Transit Gateway?

AWS Transit Gateway là một dịch vụ cho phép bạn kết nối nhiều Amazon VPC, các mạng on-premises và những tài nguyên mạng khác theo mô hình hub-and-spoke (trung tâm – vệ tinh). Có thể hình dung Transit Gateway như một trung tâm kết nối: thay vì mỗi VPC hay mạng riêng phải kết nối trực tiếp với nhau một cách phức tạp, tất cả đều kết nối về một điểm trung tâm duy nhất là AWS Transit Gateway. Từ đó, các VPC (bao gồm nhiều private subnet khác nhau) có thể giao tiếp với nhau thông qua hub này một cách đơn giản, dễ quản lý và có khả năng mở rộng cao.

Q22. Explain the role of AWS GuardDuty in enhancing cloud security

Để dễ hiểu, hãy tưởng tượng bạn đang tổ chức một bữa tiệc tại nhà và có một vị khách đáng ngờ, thực chất là một tên trộm giả dạng bạn bè. Trong lúc bạn mải vui chơi, người này lén đi khắp các phòng để dò xem có tài sản giá trị nào có thể lấy cắp. Nếu trong nhà bạn có một vệ sĩ, vệ sĩ đó sẽ nhận ra hành vi bất thường này vì người kia không quan tâm đến bữa tiệc mà chỉ chăm chú đi vào các phòng khác nhau, sau đó báo lại cho bạn. AWS GuardDuty hoạt động tương tự như vệ sĩ đó. AWS GuardDuty là một dịch vụ phát hiện mối đe dọa được quản lý, được thiết kế để nâng cao bảo mật cho môi trường AWS bằng cách liên tục giám sát và phát hiện các hành vi độc hại hoặc truy cập trái phép. Khi bạn kích hoạt GuardDuty, dịch vụ này sẽ theo dõi tất cả các tài nguyên và dịch vụ trong tài khoản AWS của bạn, hiểu được “hành vi bình thường” của chúng như mức sử dụng tài nguyên của EC2, hiệu năng của RDS hay cách các máy chủ và cơ sở dữ liệu hoạt động. Nếu GuardDuty phát hiện bất kỳ hoạt động bất thường nào, chẳng hạn như các hành vi không đúng với thói quen thông thường hoặc các dấu hiệu tấn công, nó sẽ ngay lập tức thông báo cho bạn để bạn có thể kịp thời xử lý, từ đó giúp bảo vệ hệ thống và dữ liệu của bạn một cách hiệu quả.

Q23. How do you optimize costs for a high-traffic AWS application?

Có nhiều cách để tối ưu chi phí cho một ứng dụng AWS có lượng truy cập lớn. Trước hết là right sizing tài nguyên, tức là điều chỉnh kích thước tài nguyên cho phù hợp với nhu cầu thực tế. Bạn có thể theo dõi mức sử dụng bằng AWS Cost Explorer và CloudWatch trong một khoảng thời gian như một tuần, hai tuần hoặc cả tháng để biết chi phí đang phát sinh ở đâu. Từ đó, bạn sẽ nhận ra những máy chủ hoặc dịch vụ nào ít được sử dụng hơn và giảm bớt tài nguyên cấp phát cho chúng, giúp chi phí luôn nằm trong ngân sách và không bị lãng phí. Tiếp theo, bạn có thể sử dụng Savings Plans bằng cách cam kết cho các workload có tính dự đoán trước, vì hình thức này không phù hợp với các workload khó kiểm soát hoặc biến động mạnh. Một cách khác là tận dụng Amazon

CloudFront, đây là một CDN cho phép bạn cache nội dung trên các máy chủ gần người dùng hơn, giúp giảm chi phí và tài nguyên khi phải truyền dữ liệu liên tục từ máy chủ chính. Ngoài ra, bạn có thể tối ưu chi phí cơ sở dữ liệu bằng cách sử dụng Aurora Serverless hoặc DynamoDB với chế độ on-demand, để tài nguyên chỉ được sử dụng đúng theo nhu cầu thực tế và không chiếm không gian lưu trữ không cần thiết. Cuối cùng là lựa chọn đúng lớp lưu trữ cho Amazon S3, đặc biệt với dữ liệu được truy cập thường xuyên. Trong S3 có nhiều storage class khác nhau, chẳng hạn như S3 Intelligent-Tiering, cho phép tự động tối ưu chi phí dựa trên tần suất truy cập dữ liệu, từ đó giúp giảm đáng kể chi phí lưu trữ.

Q24. What is AWS Direct Connect, and what are its benefits?

AWS Direct Connect là một dịch vụ mạng chuyên dụng cho phép doanh nghiệp thiết lập kết nối riêng tư, tốc độ cao và độ trễ thấp giữa hạ tầng tại chỗ (on-premise) như trung tâm dữ liệu, văn phòng hoặc môi trường colocation với AWS. Có thể hình dung như việc hai văn phòng trong cùng một công ty cần gửi các tệp dữ liệu mật cho nhau: nếu sử dụng dịch vụ bưu chính công cộng, dữ liệu có thể bị can thiệp hoặc rò rỉ; còn nếu dùng dịch vụ chuyển phát riêng, dữ liệu sẽ được truyền đi an toàn hơn. Tương tự, AWS Direct Connect cung cấp một đường truyền riêng thay vì đi qua Internet công cộng, giúp giảm rủi ro bảo mật, tăng tốc độ truyền dữ liệu và giảm độ trễ. Nhờ đó, doanh nghiệp có thể truyền tải dữ liệu ổn định, an toàn và hiệu quả hơn khi kết nối với các dịch vụ AWS.

Q25. How do you monitor and troubleshoot performance issues using Amazon CloudWatch?

Amazon CloudWatch là một dịch vụ của AWS cho phép bạn giám sát và theo dõi hiệu năng của tất cả các dịch vụ đang được sử dụng trong hệ thống. Có thể hình dung CloudWatch giống như một bếp ăn với nhiều đầu bếp đang làm việc, và có một bếp trưởng đứng giám sát để đảm bảo mọi món ăn được hoàn thành đúng thời gian. Mỗi món ăn đều có một mốc thời gian nhất định, nếu quá thời gian đó thì sẽ có cảnh báo để bếp trưởng kiểm tra xem vấn đề nằm ở đâu. Trong quá trình làm việc, các đầu bếp ghi lại mọi hoạt động và sự cố vào sổ nhật ký (log), nhờ đó bếp trưởng có thể xem lại để biết nguyên nhân, chẳng hạn như thiết bị không hoạt động, mất điện hay thiếu nguyên liệu, từ đó khắc phục sự cố. Tương tự, Amazon CloudWatch cho phép theo dõi hiệu năng tài nguyên và ứng dụng AWS theo thời gian thực, thiết lập các cảnh báo (alarms) khi những chỉ số như CPU hay RAM của EC2 vượt quá ngưỡng cho phép, đồng thời lưu lại CloudWatch Logs ghi nhận chi tiết mọi hoạt động của tài nguyên theo từng giây hoặc mili-giây. Khi xảy ra sự cố, bạn có thể phân tích các log này để xác định nguyên nhân và xử lý. Ngoài ra, CloudWatch còn cung cấp Dashboard giúp bạn theo dõi trực quan, theo thời gian thực tình trạng hoạt động của EC2, RDS và các tài nguyên khác, từ đó dễ dàng giám sát và tối ưu hiệu năng hệ thống.

Q26. What are the differences between Amazon Aurora and Amazon RDS?

Amazon RDS là dịch vụ cơ sở dữ liệu quan hệ đã được nhắc đến trước đó, còn Amazon Aurora có thể được xem như một phiên bản nâng cao và nhanh hơn của RDS. Có thể hình dung Amazon RDS giống như một chiếc xe bình thường, còn Amazon Aurora là một chiếc xe thể thao: cả hai đều là xe, nhưng Aurora được tối ưu để đạt hiệu năng cao hơn. Về hiệu năng, với những website nhỏ và ít biến động lưu lượng, RDS vẫn đáp ứng tốt; tuy nhiên, khi xảy ra các đợt truy cập tăng đột biến, RDS có thể bị chậm lại. Trong khi đó, Aurora được thiết kế để xử lý lưu lượng lớn, nên ngay cả khi website có traffic rất cao, Aurora vẫn duy trì khả năng truy xuất và hiển thị dữ liệu nhanh chóng, vì vậy thường được ưu tiên sử dụng hơn so với RDS trong các hệ thống lớn. Về hệ quản trị cơ sở dữ liệu hỗ trợ, Amazon RDS cung cấp nhiều lựa chọn như MySQL, PostgreSQL, MariaDB, Oracle và SQL Server, còn Amazon Aurora chỉ tập trung vào hai phiên bản tối ưu là Aurora MySQL và Aurora PostgreSQL.

Q27. What is Amazon Kinesis, and how does it support real-time data processing?

Amazon Kinesis là một dịch vụ của AWS được thiết kế để xử lý dữ liệu thời gian thực. Có thể hình dung một ví dụ như bạn đang vận hành một nền tảng game nhiều người chơi, nơi có rất nhiều người chơi cùng kết nối vào trò chơi và dữ liệu của họ liên tục được gửi về dưới dạng các luồng (stream). Vấn đề đặt ra là làm thế nào để đảm bảo toàn bộ dữ liệu trực tiếp này được thu thập, xử lý chính xác và không xảy ra gian lận hay hành vi không hợp lệ trong trò chơi. Đây chính là lúc Amazon Kinesis phát huy vai trò của mình. Amazon Kinesis cung cấp các công cụ để thu thập, quản lý và phân tích các luồng dữ liệu liên tục từ nhiều nguồn khác nhau như log ứng dụng, thiết bị IoT hay các luồng dữ liệu mạng xã hội. Dịch vụ này đặc biệt phù hợp khi cần xử lý dữ liệu theo thời gian thực hoặc dữ liệu “live”. Không chỉ dừng lại ở việc tiếp nhận dữ liệu, Amazon Kinesis còn cho phép xử lý, phân tích và thao tác trực tiếp trên các luồng dữ liệu đó, giúp hệ thống phản hồi nhanh chóng và chính xác với những gì đang diễn ra trong thời gian thực.

Q28. What are the differences between Amazon S3 storage classes?

Amazon S3 cung cấp nhiều lớp lưu trữ khác nhau để phù hợp với các nhu cầu truy cập và tối ưu chi phí. Trước hết là S3 Standard, dùng cho dữ liệu được truy cập thường xuyên. Tiếp theo là S3 Intelligent-Tiering, lớp lưu trữ giúp tự động tối ưu chi phí cho dữ liệu có tần suất truy cập không xác định hoặc thay đổi theo thời gian. Có thể hình dung như một cửa hàng ăn uống: không phải lúc nào cũng đông khách, nhưng vào những khung giờ cao điểm thì lượng truy cập tăng rất mạnh, vì vậy Intelligent-Tiering sẽ tự động điều chỉnh để tiết kiệm chi phí. Sau đó là S3 Standard-IA (Infrequent Access), dành cho dữ liệu ít được truy cập hơn nhưng vẫn cần truy xuất nhanh khi cần. S3 One Zone-IA cũng dành cho dữ liệu ít truy cập, nhưng chỉ lưu trữ trong một Availability Zone nên chi phí thấp hơn. Tiếp theo là S3 Glacier Instant Retrieval, phù hợp cho dữ liệu dài hạn, hiếm khi truy cập nhưng vẫn cần khả năng truy xuất gần như tức thì. S3 Glacier Flexible Retrieval dành cho dữ liệu lưu trữ lâu dài với thời gian truy xuất linh

hoạt hơn, thường được sử dụng khi cần tối ưu chi phí và có thể theo dõi mức sử dụng tài nguyên bằng các công cụ như AWS Cost Explorer và CloudWatch. Cuối cùng là S3 Glacier Deep Archive, lớp lưu trữ có chi phí thấp nhất, dùng cho việc lưu trữ dữ liệu trong thời gian rất dài hoặc lưu trữ số liệu lưu trữ (archive) khi bạn biết chắc rằng dữ liệu cần được giữ lại nhưng hầu như không bao giờ truy cập.

Q29. What is the role of Amazon CloudFront in improving application performance?

Amazon CloudFront là một dịch vụ CDN (Content Delivery Network) của AWS, có vai trò quan trọng trong việc cải thiện hiệu năng và độ tin cậy của các ứng dụng web. Có thể hình dung như sau: nếu website của bạn được đặt máy chủ tại Ấn Độ và người dùng truy cập chủ yếu từ khu vực đó, thì tốc độ tải sẽ rất nhanh. Tuy nhiên, khi người dùng ở các khu vực xa hơn như châu Âu (ví dụ Đức) truy cập, dữ liệu sẽ phải đi một quãng đường rất dài từ máy chủ gốc, dẫn đến thời gian tải chậm. Điều này gây bất lợi, đặc biệt trong các tình huống cần truy cập đồng thời như khi ra mắt sản phẩm hay bán merchandise, vì người dùng gần máy chủ sẽ có lợi thế hơn so với người dùng ở xa. Amazon CloudFront giải quyết vấn đề này bằng cách lưu (cache) nội dung của website tại các máy chủ CDN phân bố trên toàn cầu, đặc biệt gần với người dùng cuối. Nhờ đó, người dùng ở bất kỳ khu vực nào cũng có thể truy cập dữ liệu từ máy chủ gần nhất thay vì phải kết nối trực tiếp tới máy chủ gốc, giúp giảm độ trễ, tăng tốc độ tải trang và mang lại trải nghiệm mượt mà hơn. Nói một cách kỹ thuật, Amazon CloudFront là dịch vụ phân phối nội dung giúp cải thiện hiệu năng và độ ổn định của ứng dụng web bằng cách đưa dữ liệu đến gần người dùng hơn.

Q30. What strategies can be used to ensure disaster recovery in AWS?

Disaster Recovery là một khái niệm vô cùng quan trọng đối với bất kỳ nền tảng điện toán đám mây nào, bởi khi lưu trữ dữ liệu trên cloud, bạn thường lưu trữ khối lượng dữ liệu rất lớn, bao gồm cả dữ liệu riêng tư và quan trọng. Vì vậy, việc đảm bảo dữ liệu luôn an toàn và có thể truy cập được trong trường hợp xảy ra sự cố là điều thiết yếu. Để quá trình khôi phục thảm họa diễn ra suôn sẻ, có thể áp dụng nhiều chiến lược khác nhau. Trước hết là backup và restore, tức là thường xuyên sao lưu dữ liệu quan trọng lên Amazon S3 hoặc Amazon S3 Glacier. Tiếp theo là chiến lược pilot light, trong đó bạn chỉ duy trì một phiên bản tối thiểu của ứng dụng làm bản sao lưu, không cần lưu toàn bộ hệ thống; các cấu hình và tài nguyên có thể được định nghĩa sẵn bằng CloudFormation để khi cần có thể nhanh chóng mở rộng. Một chiến lược khác là multi-region active-active, tức là chạy ứng dụng đồng thời ở nhiều AWS Region khác nhau để nếu một Region gặp sự cố thì Region còn lại vẫn hoạt động và đảm bảo dịch vụ không bị gián đoạn. Bên cạnh đó là multi-region active-passive, trong đó các Region thụ động không hoạt động thường xuyên nhưng vẫn lưu trữ dữ liệu và sẵn sàng thay thế khi Region chính gặp sự cố. Sao chép dữ liệu (data replication) cũng rất quan trọng, giúp dữ liệu luôn được nhân bản liên tục sang các Region hoặc dịch vụ lưu trữ khác để đảm bảo tính sẵn sàng. Ngoài ra, việc áp dụng infrastructure as code thông qua AWS CloudFormation

cho phép bạn lưu lại toàn bộ cấu hình hạ tầng dưới dạng mã, nhờ đó có thể nhanh chóng triển khai lại hệ thống khi xảy ra thảm họa. Cuối cùng, giám sát và tự động hóa bằng các dịch vụ như Amazon CloudWatch và AWS GuardDuty giúp phát hiện sớm các dấu hiệu bất thường hoặc sự cố tiềm ẩn, từ đó cho phép bạn kịp thời xử lý trước khi thảm họa thực sự xảy ra.